

PHYSICS-INFORMED TRAINING AT THE OPERATOR LEVEL

Anonymous authors

Paper under double-blind review

ABSTRACT

We present an approach to train neural operators physics-informed, i.e., in a self-supervised fashion, purely relying on the governing physical equations. This enables training on infinite datasets where no sample is visited twice and removes the need to produce training data in an offline phase. While previous attempts for physics-informed training at the operator level were limited to benign problems and performed considerably worse than data-driven learning, our approach matches the gold-standard of data-driven training at a similar optimization cost. Our method is based on modifying the physics-informed loss functions to include a preconditioner such as a multigrid method to drastically reduce the ill-conditioning caused by unbounded PDE operators. We derive the improved conditioning theoretically, and illustrate the potential of the method for the Poisson, Allen-Cahn, and stationary Stokes equation, where it performs virtually identical to data-driven training at a comparable computational cost.

1 INTRODUCTION

Neural Operators Despite a century of development of numerical methods, partial differential equations (PDEs) remain expensive to solve, even more so in many-query situations, such as uncertainty quantification, optimal control, and inverse problems. The development of fast PDE solution techniques, e.g., through reduced-order/surrogate models therefore is of utmost importance. In the wake of the deep learning revolution, neural operators emerged as a powerful computational tool for surrogate modeling, drastically reducing PDE solution time in many applications. Neural operators approximate maps between infinite-dimensional spaces (operators) by a neural network, and are typically trained in a supervised fashion. They hence require a dataset that is built in an offline fashion, before they can be trained, and eventually used downstream. Data generation can constitute a substantial part of the computation and engineering effort, even more so in the context of physics foundation models that are pre-trained over an array of different PDE instances.

Physics-Informed Training Whenever the governing physics in the form of PDEs are known, neural operators can—in principle—be trained in a self-supervised fashion, relying on the physics residual or variational principles as a loss function. This approach is referred to as *physics-informed neural operator training*, and a natural idea present in the community since the inceptions of neural operators []. However, the success of physics-informed training of neural operators has remained limited to benign equations and learning tasks. More often than not, purely physics-informed learning fails to converge. In fact, supervised learning with its offline data-generation pipeline remains the gold standard for almost all applications. Nevertheless, physics-informed training offers unique

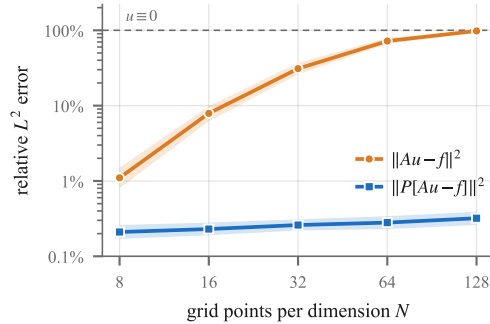


Figure 1: Adam on the plain versus the preconditioned Poisson least-squares loss, $N \times N$ grid. Placeholder numbers.

advantages, such as the training in the infinite data-limit, where no sample is revisited, and, of course, it removes the need of an expensive dataset generation process.

Ill-Conditioning of PDE Operators In this manuscript, we show that the pathologies of physics-informed neural operator learning can be attributed to a single foundational property of PDE operators: PDE operators are unbounded maps, meaning they magnify their inputs without bounds, and thus lead to severe ill-conditioning in physics-informed loss functions. To make this concrete, take the derivative operator

$$\frac{d}{dx} : C^1(0, 1) \subset L^2(0, 1) \rightarrow L^2(0, 1), \quad u \mapsto u', \quad (1)$$

which amplifies oscillations: $x \mapsto \sin(\omega x)$ is bounded in $L^2(0, 1)$ independent of $\omega \in \mathbb{R}$, but its derivative $x \mapsto \omega \cos(\omega x)$ grows without bound in $L^2(0, 1)$ when $\omega \rightarrow \infty$. This phenomenon is inherent in all PDE operators. In the case of Poisson’s equation, and for a linear model that is the linear combination of a given basis set $\phi_1, \dots, \phi_n$, the physics-informed least-squares loss¹ on a single data point f takes the form

$$L_{\text{LS}}(u) = \frac{1}{2} \|Au - f\|^2, \quad A_{ij} = \int_{\Omega} \nabla \phi_j \nabla \phi_i \, dx, \quad (2)$$

where $A \in \mathbb{R}^{N \times N}$ is the stiffness matrix. The condition number of A scales like h^{-2} for standard bases, where h is the characteristic mesh-size. Conditioning thus deteriorates under mesh refinement, as higher oscillating frequencies can be resolved, the same manifestation of the unbounded nature of PDE operators stressed in 1. Note that $\nabla^2 L_{\text{LS}} = A^2$, hence the condition number governing the convergence speed of gradient descent (GD) scales like h^{-4} . Figure 1 illustrates that this ill-conditioning is so severe that training with first-order methods like GD and Adam becomes infeasible already for very coarse meshes, even in the benign case of a linear ansatz (no neural network at all). In the remainder of this article, we show that precisely this ill-conditioning renders physics-informed neural operator learning infeasible.

Preconditioned Physics-Informed Training To counter the ill-conditioning of the least-squares loss equation 2, we rely on *preconditioning*—precisely the tool the numerical analysis community uses to efficiently employ iterative methods in ill-conditioned settings in classical scientific computing. Abstractly, we *modify the least-squares loss* to include an invertible preconditioner P and a positive definite and symmetric weighting matrix B

$$L_{\text{PLS}}(u) = \frac{1}{2} \|P[Au - f]\|_B^2, \quad (3)$$

We stress that this construction is agnostic to the neural operator architecture employed and adds zero overhead to the inference once training finishes as it only influences the loss function. On a conceptual level, the preconditioned loss equation 3 changes the norm in which the error is measured. For any given P we can recast the

$$L_{\text{PLS}}(u) = \frac{1}{2} \|P[Au - f]\|_B^2 = \frac{1}{2} (PAe, PAe)_B$$

with the error $e = u - A^{-1}f$. This means P maps Ae into a different space and thereby improves conditioning, and B measures the error in this space in an appropriate manner. The choice $P = A^{-1}$ and $B = I^2$ recovers data-driven training and reduces the condition number of $\nabla^2 L_{\text{PLS}}$ to unity. In practice, this is realized through fast approximate inverses $P \approx A^{-1}$, e.g., via multigrid methods []. Note that $P \approx A^{-1}$ is not the only choice; for saddle point problems, we alternatively employ block-diagonal preconditioners. Additionally, whenever applicable, we investigate energy formulations, as these are naturally less ill-conditioned.

1.1 MAIN CONTRIBUTIONS

The main contributions are

¹If the equation admits, energy formulations (variational principles) are an attractive alternative.

²Or alternatively, $B = M$, where M is the mass matrix.

- **Efficient physics-informed neural operator training.** We demonstrate that preconditioned physics-informed neural operator training allows data-free training of neural operators at comparable cost and accuracy of the gold standard of data-driven training. Our approach vastly outperforms previous approaches and is demonstrated for the Poisson, Allen-Cahn, and Stokes equation.
- **Flexibility and inference speed.** Our method can be combined with any neural operator architecture and once trained, adds no additional cost to the inference.
- **Training at infinite data limit.** Physics-informed training allows to practically approximate the infinite data limit, where no sample is visited twice. We demonstrate empirically that this is a favorable training paradigm for neural operators.

1.2 RELATED LITERATURE

- Discussion of physics-informed training in general, in particular for PINNs and NQS.
- Paper of Wolfgang and Markus on preconditioning.
- OG paper on physics-informed neural operators.
- Papers on suboptimal fixes for physics-informed neural operator training.

2 PHYSICS-INFORMED TRAINING OF NEURAL OPERATORS

Our goal is to approximate an operator $\mathcal{G} : \mathcal{F} \rightarrow \mathcal{U}$ between function spaces $\mathcal{U}$ and $\mathcal{F}$. We aim to approximate $\mathcal{G}$ by a neural network $\mathcal{G} \approx \mathcal{G}_\theta$. Neural operators are a special class of neural networks well-suited to the infinite-dimensional nature of the function spaces appearing in PDE applications. However, the concrete neural network structure of $\mathcal{G}_\theta$ is secondary for our proposed training methodology. A simple but crucial observation we make, is that we can always write the neural operator output in the *interpolated form*

$$\mathcal{G}_\theta(f) = \sum_{i=1}^n \mathbf{u}(\theta, f)_i \phi_i, \quad (4)$$

where $\mathbf{u}(\theta, f) \in \mathbb{R}^n$ are coefficients depending on the trainable weights and the functional input f . The functions $\phi_1, \dots, \phi_n$ are a fixed basis and span a finite-dimensional function space $\mathcal{U}_h$, where h denotes a characteristic mesh-size³. Typical choices are finite element spaces, like piecewise polynomial spaces on a triangulated computational domain. It is crucial to stress that $\phi_1, \dots, \phi_n$, and thus $\mathcal{U}_h$ is a modeling choice, and a standard neural operator architecture will not automatically map into $\mathcal{U}_h$. However, we can always interpolate into $\mathcal{U}_h$. This is a typically sparse, non-trainable operation that adds negligible computational overhead and we will from now on assume that our neural operator is the form described in equation 4.

2.1 PHYSICS-INFORMED LOSS FUNCTIONS

For simplicity of exposition, we will work in the setting of linear PDEs that can be formulated in terms of bilinear forms, and we will restrict the discussion to learning the solution operator of such PDEs, although our experimental evaluation is broader. The operator learning problem is to find $u \in \mathcal{U}$ such that for given $f \in \mathcal{F}$ it holds

$$a(u, v) = f(v), \quad \text{for all } v \in \mathcal{U}. \quad (5)$$

Here, a is a bounded and symmetric bilinear form on $\mathcal{U}$, and the setting implies $\mathcal{F} = \mathcal{U}^*$. We assume all spaces to be Hilbert spaces. The solution operator is $\mathcal{G} : f \mapsto u$, where u solves equation 5. The weak equation can be written as a matrix equation, once u is approximated in $\mathcal{U}_h$, and takes the form

$$A\mathbf{u} = \mathbf{f}, \quad \text{with } A_{ij} = a(\phi_j, \phi_i), \quad \mathbf{f}_i = f(\phi_i).$$

From this equation, we derive the single sample physics-informed loss function via a least-squares construction

$$L_{\text{LS}}(\theta) = \frac{1}{2} \|A\mathbf{u}(\theta, f) - \mathbf{f}\|^2. \quad (6)$$

³Or more general, a quantification of how “fine” the space $\mathcal{U}_h$ is.

For analysis purposes, we will frequently ignore the nonlinear parametrization $\mathbf{u}(\theta, f)$ and work in the Euclidean space of $\mathbf{u} \in \mathbb{R}^n$ directly. We indicate this by writing $L_{\text{LS}}(u) = \frac{1}{2} \|A\mathbf{u} - \mathbf{f}\|^2$ with the understanding that everything is in the variable $\mathbf{u}$. If a is coercive and symmetric, an energy formulation is an attractive alternative to equation 5. In this case, we can find the solution u via a variational principle as the minimizer of the energy $\frac{1}{2}a(v, v) - f(v)$. The loss function to train the neural operator is, again on a single sample

$$L_{\text{Var}}(\theta) = \frac{1}{2} \mathbf{u}(\theta, f)^\top A \mathbf{u}(\theta, f) - \mathbf{f}^\top \mathbf{u}(\theta, f). \quad (7)$$

2.2 PRECONDITIONING

The least-square loss 6, although faithfully recovering the solution of $A\mathbf{u} = \mathbf{f}$ whenever A is bijective, is unworkable in practice due to its ill-conditioning. This can be seen by computing the Hessian $\nabla_u^2 L_{\text{LS}} = A^\top A$. In the representative scenario of the Poisson equation [], the condition number $\kappa(A^\top A)$ scales like h^{-4} , and makes iterative methods inefficient or, due to round-off errors, entirely useless. To counter this effect, we propose the preconditioned least-squares loss

$$L_{\text{PLS}}(u) = \frac{1}{2} \|P[A\mathbf{u} - \mathbf{f}]\|_B^2, \quad (8)$$

where $P \in \mathbb{R}^{n \times n}$ is an invertible matrix, and $B \in \mathbb{R}^{n \times n}$ is symmetric and positive definite, hence induces an inner product. The choice of P and B is intuitively coupled: P transforms the range space of A thereby improving conditioning, and B should induce a meaningful norm on the transformed space. To assess conditioning of L_{PLS} , we compute the Hessian with respect to $\mathbf{u}$

$$\nabla_u^2 L_{\text{PLS}}(u) = A^\top P^\top B P A. \quad (9)$$

We discuss a few choices of P and A .

2.2.1 APPROXIMATE INVERSE PRECONDITIONERS

We refer to the choice $P \approx A^{-1}$ and $B = I$ or a similarly well-conditioned matrix, like the mass matrix $M_{ij} = (\phi_i, \phi_j)_{L^2}$ as the approximate inverse setting. As proven in Lemma [] in the Appendix, this yields

$$\kappa(PA)^2 \approx \frac{\kappa(PA)^2}{\kappa(B)} \leq \kappa(A^\top P^\top B P A) \leq \kappa(PA)^2 \kappa(B) \approx \kappa(PA)^2$$

and in the case of a well conditioned matrix B , the conditioning is determined by the accuracy of the approximation $P \approx A^{-1}$. The approximate inverse setting is only practical when the application of P is computationally efficient. For elliptic equations, multigrid methods provide such a preconditioner P at the optimal complexity $\mathcal{O}(n)$, see [].

2.2.2 BLOCK-DIAGONAL PRECONDITIONERS

We assume the following block structure for A , $\mathbf{u}$ and $\mathbf{f}$

$$A = \begin{pmatrix} K & D^\top \\ D & 0 \end{pmatrix}, \quad \mathbf{u} = \begin{pmatrix} \mathbf{w} \\ \mathbf{p} \end{pmatrix}, \quad \mathbf{f} = \begin{pmatrix} \mathbf{g} \\ 0 \end{pmatrix}.$$

We assume that K is symmetric and positive definite, and that $D^\top$ is injective. In this situation the symmetric system A is indefinite and invertible—a well-posed discretization of a saddle point problem, as arises from Stokes equation. The Schur complement is $S = DK^{-1}D^\top$, and the classical choice of block-diagonal preconditioner is

$$P \approx \text{diag}(K^{-1}, S^{-1}),$$

where in applications the block inverses can frequently be handled by multigrid methods. The key property of the preconditioner is that PA has exactly three eigenvalues $\{1, \frac{1+\sqrt{5}}{2}, \frac{1-\sqrt{5}}{2}\}$. However, PA remains ill-conditioned in Euclidean norm as it is a non-normal matrix, and in particular, P is not an approximate inverse of A . To leverage P in the context of the least-squares loss, we use it to measure the norm of the residual (we use it as B in equation 3), i.e., the loss becomes

$$\frac{1}{2} \|A\mathbf{u} - \mathbf{f}\|_P^2 = \frac{1}{2} \|K\mathbf{w} + D^\top \mathbf{p} - \mathbf{g}\|_{K^{-1}}^2 + \frac{1}{2} \|D\mathbf{p}\|_{S^{-1}}^2.$$

As A is symmetric, its Hessian is APA , and Lemma [] shows

$$\kappa(APA) \leq \kappa(P) \left(\frac{\max |\lambda(PA)|}{\min |\lambda(PA)|} \right)^2.$$

Hence conditioning is governed by $\kappa(P)$, which is $\mathcal{O}(h^{-2})$ for the prototypical case of Stokes equation, hence ill-conditioning remains, but is drastically improved from the $\mathcal{O}(h^{-4})$ scaling without preconditioner. We discuss further preconditioning options with the block-diagonal preconditioner in Appendix [], but stress that all lead to worse outcomes. Finally, we mention that for Stokes equation monolithic multigrid methods exist, and these fall under the approximate inverse setting.

2.2.3 PRECONDITIONING ENERGY FORMULATIONS

Whenever the form a is symmetric and coercive, we can use the loss function $L_{\text{Var}}(\mathbf{u}) = \frac{1}{2} \mathbf{u}^\top A \mathbf{u} - \mathbf{f}^\top \mathbf{u}$. Its Hessian in $\mathbf{u}$ is $\nabla_u^2 L_{\text{Var}} = A$, hence its conditioning is the square root of L_{LS} ; and in the prototypical setting of the Poisson equation, it is $\mathcal{O}(h^{-2})$. Under the restrictive assumption that P and A commute and P is symmetric and positive definite, we can define the preconditioned variational loss

$$L_{\text{PVar}}(\mathbf{u}) = \frac{1}{2} (\mathbf{u}, A\mathbf{u})_P - (\mathbf{f}, \mathbf{u})_P.$$

The critical points $\nabla_u L_{\text{PVar}}(\mathbf{u}) = 0$ are characterized by $PA\mathbf{u} = P\mathbf{f}$, and thus the minimizer of L_{Var} is unchanged under this transformation. The Hessian is $\nabla_u^2 L_{\text{PVar}}(\mathbf{u}) = PA$. Preconditioners that commute with A are e.g. polynomials in A like []. In practice, we *violate the requirement* $PA = AP$ and merely modify the gradient $\nabla_u L_{\text{Var}}(\mathbf{u}) = A\mathbf{u} - \mathbf{f}$ to be $P[A\mathbf{u} - \mathbf{f}]$ in the backward pass. Empirically, the choice $P \approx A^{-1}$ via a multigrid method performs reasonable although commutativity is not exactly satisfied.

2.2.4 PRECONDITIONING THROUGH ARCHITECTURE MODIFICATION

For both L_{LS} and L_{Var} we can—in principle—achieve preconditioning by a transformation with P : $\tilde{L}_{\text{LS}}(\mathbf{u}) = L_{\text{LS}}(P\mathbf{u})$. This yields for $B = I$ for simplicity

$$\nabla_u^2 \tilde{L}_{\text{LS}}(\mathbf{u}) = P^\top A^\top AP,$$

and similarly $\nabla_u^2 \tilde{L}_{\text{Var}}(\mathbf{u}) = P^\top AP$. However, this construction requires P to be evaluated at inference time as a part of the forward pass of the model, which is why we do not develop this idea further in this manuscript.

3 NUMERICAL RESULTS

3.1 POISSON EQUATION

3.2 ALLEN-CAHN EQUATION

3.3 STOKES EQUATION

4 CONCLUSION

AI USE STATEMENT

(This section is **required** and does not count toward the page limit.)

In this work, we used generative AI tools for software implementation and language refinement. We have not used generative AI tools for drafting the manuscript, and [the rest of the required disclosure tasks] are not applicable to this work. Additionally, we used generative AI tools for [tasks with recommended disclosure]. We have reviewed all AI-assisted work. LLM-generated code was verified and tested for correctness by the authors. We take responsibility for the final content of this work, including text, claims or artifacts produced with the aid of generative AI.

ETHICS STATEMENT

(This section is **recommended** and does not count toward the page limit.)

If authors feel that their paper submission raises questions regarding the Code of Ethics, they are encouraged to include a paragraph of Ethics Statement (at the end of the main text before references) to address potential concerns where appropriate. Topics include, but are not limited to, studies that involve human subjects, practices to data set releases, potentially harmful insights, methodologies and applications, potential conflicts of interest and sponsorship, discrimination/bias/fairness concerns, privacy and security issues, legal compliance, and research integrity issues (e.g., IRB, documentation, research ethics). This statement should not be more than 1 page.

REPRODUCIBILITY STATEMENT

(This section is **recommended** and does not count toward the page limit.)

It is important that the work published in ICLR is reproducible. Authors are strongly encouraged to include a paragraph-long Reproducibility Statement at the end of the main text (before references) to discuss the efforts that have been made to ensure reproducibility. This paragraph should not itself describe details needed for reproducing the results, but rather reference the parts of the main paper, appendix, and supplemental materials that will help with reproducibility. For example, for novel models or algorithms, a link to an anonymous downloadable source code can be submitted as supplementary materials; for theoretical results, clear explanations of any assumptions and a complete proof of the claims can be included in the appendix; for any datasets used in the experiments, a complete description of the data processing steps can be provided in the supplementary materials. Each of the above are examples of things that can be referenced in the reproducibility statement.

AUTHOR CONTRIBUTIONS

If you'd like to, you may include a section for author contributions as is done in many journals. This is optional and at the discretion of the authors.

ACKNOWLEDGMENTS

Use unnumbered third level headings for the acknowledgments. All acknowledgments, including those to funding agencies, go at the end of the paper.

REFERENCES

A APPENDIX

You may include other additional sections here.